

Verified Functional Programming in Lean 4

Lecture 19

Section 1

Preliminaries

Definitional equality

Two terms are **definitionally equal** ($\equiv$) when Lean's kernel can reduce both to a common normal form. The kernel checks this silently during type-checking — you never write a proof.

This is *not* just β -reduction. The kernel uses four reduction rules:

Rule	What it does	Example
β	apply a λ	$(\lambda x. x + 1) 3 \rightarrow 3 + 1$
δ	unfold a definition	not $b \rightarrow \text{match } b \dots$
ι	match on a constructor	match true $\dots \rightarrow$ false
ζ	substitute a let	let $x := 3$; $x + 1 \rightarrow 3 + 1$

(Plus η for functions.) Two terms are definitionally equal when the kernel, using all of these rules, can reduce them to the same thing.

Definitional vs. propositional equality

Propositional equality ($=$) is the type $a = b$. A proof of $a = b$ is a term of that type. Every equation you prove in Lean is a propositional equality.

Definitional equality ($\equiv$) is a *special case*: a and b reduce to the same normal form. When this happens, `refl` (`Eq.refl`) is a proof of $a = b$ — the kernel verifies the reduction silently.

So definitional equality gives you propositional equality for free. The converse is not true: some propositional equalities are *not* definitional, and those require real work.

Recall: Eq

In Lecture 18 we defined equality as an inductive type. In Lean, it is called `Eq`:

```
inductive Eq {A : Type} (a : A) : A → Prop where
  | refl : Eq a a
```

There is only one constructor: `refl`, whose type is `Eq a a` — a proof that a equals itself.

Lean provides syntactic sugar so you rarely write `Eq` directly:

Full form	Notation
<code>Eq a b</code>	<code>a = b</code>
<code>Eq.refl</code>	<code>refl</code>

So when you see `a = b` in Lean, it is the type `Eq a b`. And when you write `refl`, you are supplying the constructor `Eq.refl` as a proof.

When `rfl` works (and when it does not)

Since `Eq.refl` has type `Eq a a`, it can only prove goals of the form $a = a$. When you offer `rfl` as a proof of $a = b$, the kernel must check that $a = b$ is $a = a$ — that is, it must verify $a \equiv b$ by reducing both sides.

If the kernel reduces a and b to the same normal form, it accepts `rfl`. If it cannot, `rfl` is rejected. The terms might still be propositionally equal, but the kernel cannot see it.

Two examples

To make this concrete, consider Lean's built-in addition on `Nat`. It is defined by pattern matching on the **second** argument:

```
def Nat.add (m : Nat) : Nat → Nat
| .zero    ⇒ m
| .succ k  ⇒ .succ (Nat.add m k)
```

This definition determines which equalities are definitional and which are not:

```
-- Definitional:  unfolds add,  matches .zero,
-- returns m. Both sides reduce to m.
example (m : Nat) : m + 0 = m := rfl          --

-- Not definitional:  unfolds add, but  is stuck
-- (m is a variable, not a constructor). rfl fails.
example (m : Nat) : 0 + m = m := rfl          -- ERROR
```

The second equation is still *true* — but the kernel cannot see it. Proving it requires induction, which we will do shortly.

Section 2

Part I: Booleans

Our own Booleans

We will build everything from scratch, in our own namespace. This way there is no magic: every definition is ours.

```
namespace Lec

inductive Bool : Type where
| true  : Bool
| false : Bool
```

This is an inductive type with two constructors and no arguments. Every element of `Bool` is either `true` or `false`, and nothing else.

Defining functions by pattern matching

```
def not : Bool → Bool
| .true  ⇒ .false
| .false ⇒ .true

def and : Bool → Bool → Bool
| .true,  b ⇒ b
| .false, _ ⇒ .false

def or : Bool → Bool → Bool
| .true,  _ ⇒ .true
| .false, b ⇒ b
```

Each definition is exhaustive: one branch per constructor. The kernel checks this.

```
#reduce not .true      -- .false  
#reduce and .true .false -- .false  
#reduce or .false .true  -- .true
```

`#reduce` asks the kernel to normalize the term. For `not .true`: first a δ -step unfolds `not` to its match expression, then an ι -step sees the constructor `.true` and selects the first branch, returning `.false`.

These are not “running a program.” The kernel is reducing a term to normal form inside the type theory.

Our first theorem

Claim: $\text{not } (\text{not } b) = b$ for all b .

```
theorem not_not (b : Bool) : not (not b) = b := by
  cases b
  • -- goal: not (not .true) = .true
    rfl
  • -- goal: not (not .false) = .false
    rfl
```

`cases b` splits into one subgoal per constructor. In each branch, b is replaced by a specific constructor. Then two δ/ι cycles fire:
 $\text{not true} \rightarrow_{\delta} \text{match true} \dots \rightarrow_{\iota} \text{false}$, and then $\text{not false} \rightarrow_{\delta} \rightarrow_{\iota} \text{true}$. Both sides are now definitionally equal, and `rfl` closes the goal.

The same proof as a term

The tactic proof above builds this kernel term:

```
theorem not_not' (b : Bool) : not (not b) = b :=  
  match b with  
  | .true  => rfl    -- Eq.refl Bool.true  
  | .false => rfl    -- Eq.refl Bool.false
```

`match b` applies the recursor for `Bool`. In each branch the type checker reduces and sees both sides are identical. `rfl` is `Eq.refl`.

Tactic mode and term mode produce the same kernel term. We will use both freely.

Another theorem: and is commutative

```
theorem and_comm (a b : Bool) : and a b = and b a := by
  cases a <;> cases b <;> rfl
```

`cases a <;> cases b` splits both variables, producing four subgoals:

<i>a</i>	<i>b</i>	goal reduces to
true	true	true = true
true	false	false = false
false	true	false = false
false	false	false = false

Each is closed by `rfl`. Proofs over `Bool` are often just exhaustive case analysis.

Exercises (try these yourself)

- 1 Prove `or_comm`: $\text{or } a \ b = \text{or } b \ a$.
- 2 Prove `and_assoc`: $\text{and } (\text{and } a \ b) \ c = \text{and } a \ (\text{and } b \ c)$.
- 3 Define `xor` : $\text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool}$ and prove that $\text{xor } b \ b = \text{false}$.

All of these can be proved by cases and `rf1`. Why? Because `Bool` has finitely many elements, so exhaustive checking works.

This strategy will **not** work for `Nat` or `List`: those types are infinite. We will need induction.

Section 3

Part II: Natural Numbers

Defining the natural numbers

```
inductive N : Type where  
  | zero : N  
  | succ : N → N
```

Two constructors: `zero` takes no arguments; `succ` takes one recursive argument. Every natural number is either `zero` or `succ  $k$` for some k .

This is the type `Nat` from Lectures 17–18, now as a Lean definition.

Defining addition

```
def add : N → N → N
| m, .zero    ⇒ m
| m, .succ k  ⇒ .succ (add m k)
```

Addition recurs on the **second** argument. Lean checks that k is structurally smaller than $.succ\ k$, guaranteeing termination.

Consequence: $add\ m\ .zero$ reduces by $\delta + \iota$ (unfold, then match on zero). But $add\ .zero\ m$ does **not** reduce when m is a variable — the ι -step is stuck.

Seeing computation

```
#reduce add (.succ .zero) (.succ (.succ .zero))  
-- .succ (.succ (.succ .zero))      (i.e., 1 + 2 = 3)
```

Trace:

$$\begin{aligned}\text{add } (S\ 0) (S\ (S\ 0)) &\rightarrow S\ (\text{add } (S\ 0) (S\ 0)) \\ &\rightarrow S\ (S\ (\text{add } (S\ 0) 0)) \\ &\rightarrow S\ (S\ (S\ 0))\end{aligned}$$

Each step is a δ -step (unfold `add`) followed by an ι -step (the match sees `succ  $k$` and selects the second branch). The last step matches `zero` and selects the first branch.

A definitional equality

Since `add` pattern-matches on the second argument, the equation $m + 0 = m$ holds *definitionally*: a δ -step unfolds `add`, then an ι -step matches zero against the first branch and returns m .

```
theorem add_zero (m : N) : add m .zero = m :=  
  rfl
```

No induction needed. The kernel does the work silently.

Tactic mode

So far every proof has been a **term**: we wrote `rf1` directly. But some proofs require multiple steps. Lean provides **tactic mode**, entered with the keyword `by`:

```
theorem foo : ... := by
  tactic
  tactic
  ...
```

A tactic transforms the current **goal** (the type we need to inhabit). Each tactic either closes the goal or produces simpler subgoals. When all goals are closed, Lean assembles the tactics into a kernel term.

The tactics we need

```
induction m with | zero => ... | succ k ih => ...
```

Applies the recursor for the type of m . Produces one subgoal per constructor. In the recursive case ($\text{succ } k \text{ ih}$), you get an **induction hypothesis** ih for the smaller value k .

```
show T
```

Replaces the current goal with T , which must be definitionally equal to the original goal. Useful to make the reduced form of the goal explicit after δ/ι -steps have fired.

`rw [h]` is arguably the most important tactic in Lean.

Given `h : a = b`, the tactic `rw [h]` searches the goal for an occurrence of `a` (the left-hand side of `h`) and replaces it with `b` (the right-hand side):

```
-- goal:      succ (add zero k) = succ k
-- h :         add zero k = k
-- after rw [h]:
-- goal:      succ k = succ k           (closed by rfl)
```

The tactic found `add zero k` in the goal, replaced it with `k`, and the two sides became identical.

rw: chaining and direction

`rw [h1, h2, h3]` applies the rewrites **in order**, left to right. Each rewrite transforms the goal, and the next one operates on the result. If after all rewrites the goal is definitionally equal on both sides, it closes automatically.

By default, `rw [h]` rewrites left-to-right: it replaces the LHS of `h` with the RHS. To go in the other direction, use the back arrow:

```
-- h : a = b
-- rw [h]      replaces a with b in the goal
-- rw [← h]    replaces b with a in the goal
```

Example: if the goal is `k = add zero k` and `h : add zero k = k`, then `rw [h]` would try to replace `add zero k` with `k`, giving `k = k`. But `rw [← h]` would replace `k` with `add zero k` — probably not what you want. The direction matters.

A propositional equality: the first induction

The equation $0 + m = m$ does *not* hold definitionally: after the δ -step unfolding `add`, the ι -step is stuck because the second argument is m (a variable, not a constructor). We need induction.

```
theorem zero_add (m : N) : add .zero m = m := by
  induction m with
  | zero      ⇒ rfl
  | succ k ih ⇒
    -- ih : add .zero k = k
    -- goal: add .zero (.succ k) = .succ k
    -- second arg is .succ k (a constructor), so fires:
    -- add .zero (.succ k) reduces to .succ (add .zero k)
    show N.succ (add .zero k) = N.succ k
    rw [ih]
```

What happened

Base case ($m = \text{zero}$): the goal is $\text{add } 0 \ 0 = 0$. Both sides reduce to zero.

Step case ($m = \text{succ } k$): the goal is $\text{add } 0 \ (\text{succ } k) = \text{succ } k$.

- The LHS reduces: $\text{add } 0 \ (\text{succ } k) \rightarrow_{\delta/\iota} \text{succ } (\text{add } 0 \ k)$.
- We have the induction hypothesis: $\text{add } 0 \ k = k$.
- Rewriting gives $\text{succ } k = \text{succ } k$, which is `rfl`.

This is the pattern: δ/ι -reduction peels off one constructor, the IH handles the recursive part.

Successor on the right

```
theorem add_succ (m k : N)
  : add m (.succ k) = .succ (add m k) :=
  rfl
```

This is definitional: the δ -step unfolds `add`, and the ι -step matches `succ k` against the second branch.

Successor on the left

```
theorem succ_add (m k : N)
  : add (.succ m) k = .succ (add m k) := by
  induction k with
  | zero      ⇒ rfl
  | succ k ih ⇒
    -- ih : add (.succ m) k = .succ (add m k)
    -- goal: add (.succ m) (.succ k) = .succ (add m (.succ k))
    -- LHS reduces: .succ (add (.succ m) k)
    -- RHS reduces: .succ (.succ (add m k))
    show N.succ (add (.succ m) k) = N.succ (N.succ (add m k))
    rw [ih]
```

Same pattern: induct on the argument that add recurs on, use $\delta/\iota + \text{IH}$.

Commutativity

With `zero_add` and `succ_add` in hand:

```
theorem add_comm (m n : N) : add m n = add n m := by
  induction n with
  | zero ⇒
    -- goal: add m .zero = add .zero m
    rw [add_zero, zero_add]
  | succ k ih ⇒
    -- ih : add m k = add k m
    -- goal: add m (.succ k) = add (.succ k) m
    rw [add_succ, succ_add, ih]
```

Each `rw` step applies one of our earlier lemmas. The proof builds a chain of equalities from simple pieces.

Associativity

```
theorem add_assoc (m n k : N)
  : add (add m n) k = add m (add n k) := by
  induction k with
  | zero      ⇒ rfl
  | succ k ih ⇒
    -- goal: add (add m n) (.succ k) = add m (add n (.succ k))
    -- both sides reduce (add_succ):
    show N.succ (add (add m n) k) = N.succ (add m (add n k))
    rw [ih]
```

Induct on k (the argument `add` recurs on). The base case is definitional; the step follows from one rewrite.

Multiplication

```
def mul : N → N → N
| _, .zero    ⇒ .zero
| m, .succ k  ⇒ add (mul m k) m
```

$m \times 0 = 0$ and $m \times (k + 1) = m \times k + m$. Again, recursion on the second argument.

```
#reduce mul (.succ (.succ .zero)) (.succ (.succ (.succ .zero)))
-- six succs: 2 * 3 = 6
```

A theorem about multiplication

```
theorem mul_zero (m : N) : mul m .zero = .zero :=
  rfl

theorem zero_mul (m : N) : mul .zero m = .zero := by
  induction m with
  | zero      ⇒ rfl
  | succ k ih ⇒
    -- goal: mul .zero (.succ k) = .zero
    -- reduces to: add (mul .zero k) .zero = .zero
    show add (mul .zero k) .zero = .zero
    rw [add_zero, ih]
```

The same asymmetry: $m \times 0 = 0$ is definitional; $0 \times m = 0$ requires induction.

Section 4

Part III: Lists

Defining lists

```
inductive List (A : Type) : Type where
| nil   : List A
| cons  : A → List A → List A
```

A is a **parameter**: it is the same in every constructor. Every list is either `nil` or `cons  $x$   $xs$` for some head x and tail xs .

Length

```
def length : List A → N
| .nil      ⇒ .zero
| .cons _ xs ⇒ .succ (length xs)
```

Recursion on the list. Lean checks that `xs` is structurally smaller than `.cons _ xs`.

```
#reduce length (.cons 1 (.cons 2 (.cons 3 (.nil : List Nat))))
-- .succ (.succ (.succ .zero))
```

Append

```
def append : List A → List A → List A
| .nil,      ys ⇒ ys
| .cons x xs, ys ⇒ .cons x (append xs ys)
```

Recursion on the first list. This means `append .nil ys` reduces to `ys` (definitional), but `append xs .nil` does not.

Append and nil

```
theorem append_nil (ys : List A) : append .nil ys = ys := rfl

theorem nil_append (xs : List A) : append xs .nil = xs := by
  induction xs with
  | nil          ⇒ rfl
  | cons x xs ih ⇒
    -- goal: append (.cons x xs) .nil = .cons x xs
    -- reduces to: .cons x (append xs .nil) = .cons x xs
    show List.cons x (append xs .nil) = List.cons x xs
    rw [ih]
```

The same pattern as $m + 0 = m$ vs. $0 + m = m$: one is definitional, the other requires induction.

A new tactic: `simp only`

The next proof needs a tactic we have not seen yet. Recall that `rw` works on the **syntactic** form of the goal: if the goal says `length (append .nil ys)`, then `rw` sees exactly that, even though the kernel knows it equals `length ys`. We need a way to *unfold* definitions in the goal before rewriting.

`simp only [f, g]` repeatedly applies the defining equations of `f` and `g` to simplify the goal. For a function defined by pattern matching, these equations are the branches:

```
-- append has two equations:
--   append .nil ys          = ys
--   append (.cons x xs) ys = .cons x (append xs ys)
-- so simp only [append] applies these wherever they match.
```

Unlike `rw`, which applies one equation once, `simp only` keeps applying until nothing changes. The `only` restricts it to the lemmas you list — without `only`, `simp` uses a large built-in set.

Length distributes over append

```
theorem length_append (xs ys : List A)
  : length (append xs ys) = add (length xs) (length ys) := by
  induction xs with
  | nil =>
    -- goal: length (append .nil ys) = add (length .nil) (
    length ys)
    simp only [append]
    -- unfolds append .nil ys to ys:
    --      length ys = add (length .nil) (length ys)
    simp only [length]
    -- unfolds length .nil to .zero:
    --      length ys = add .zero (length ys)
    rw [zero_add]
  | cons x xs ih =>
    -- goal: length (append (.cons x xs) ys)
    --      = add (length (.cons x xs)) (length ys)
    simp only [append]
    -- unfolds append (.cons x xs) ys to .cons x (append xs ys)
    simp only [length]
```

Append is associative

```
theorem append_assoc (xs ys zs : List A)
  : append (append xs ys) zs
  = append xs (append ys zs) := by
  induction xs with
  | nil           ⇒ rfl
  | cons x xs ih ⇒
    -- reduces to:
    -- .cons x (append (append xs ys) zs)
    -- = .cons x (append xs (append ys zs))
    show List.cons x (append (append xs ys) zs)
      = List.cons x (append xs (append ys zs))
    rw [ih]
```

Same structure: induct on the argument that `append` recurs on, δ/ι -reduction peels off the constructor, and the IH does the rest.


```
def rev : List A → List A
| .nil      ⇒ .nil
| .cons x xs ⇒ append (rev xs) (.cons x .nil)
```

This is the naive quadratic reverse. We can prove it is an involution:
 $\text{rev} (\text{rev } xs) = xs$.

But this proof is harder. It needs a lemma about how `rev` distributes over `append`.

The key lemma

```
theorem rev_append (xs ys : List A)
  : rev (append xs ys)
  = append (rev ys) (rev xs) := by
  induction xs with
  | nil =>
    simp only [append, rev]
    -- goal: rev ys = append (rev ys) .nil
    rw [nil_append]
  | cons x xs ih =>
    simp only [append, rev]
    rw [ih, append_assoc]
```

Reverse is an involution

```
theorem rev_rev (xs : List A) : rev (rev xs) = xs := by
  induction xs with
  | nil          ⇒ rfl
  | cons x xs ih ⇒
    -- goal: rev (rev (.cons x xs)) = .cons x xs
    -- rev (.cons x xs) = append (rev xs) [x]
    -- so: rev (append (rev xs) [x]) = .cons x xs
    simp only [rev]
    rw [rev_append]
    -- goal: append (rev [x]) (rev (rev xs)) = .cons x xs
    -- rev [x] reduces to [x]
    simp only [rev, append]
    rw [ih]
```

The proof has more steps but the method is the same: unfold definitions, apply lemmas, use the IH. Each step is mechanical.

Section 5

Part IV: The Method

The pattern

Every proof in this lecture followed the same method:

- 1 **Induct** on the argument that the function pattern-matches on.
- 2 **Base case:** δ/ι -reduction makes both sides identical. Close with `rf1`.
- 3 **Step case:** δ/ι -reduction peels off one constructor. The induction hypothesis handles the recursive part. Close with `rw [ih]`.

This is the core technique of verified functional programming: the structure of the proof follows the structure of the function.

Why this works

The reason is not accidental. An inductive type is defined by its constructors. A function on that type is defined by pattern matching on those constructors. A proof about that function proceeds by induction, which splits into cases — one per constructor.

Program structure = proof structure.

This is the payoff of CIC: the same mechanism (inductive types + recursion) gives us both computation and reasoning.

What we built today

Type	Functions	Theorems	Method
Bool	not, and, or	not_not, and_comm	cases + rfl
Nat	add, mul	zero_add, add_comm, add_assoc, zero_mul	induction
List	length, append, rev	length_append, append_assoc, rev_rev	induction + lemmas

What we have not yet needed

- **Dependent types** (vectors, Fin, indexed families)
- **The Prop/Type distinction** and the elimination restriction
- **Tactics beyond the basics** (simp, omega, custom simp lemmas)
- **Type classes** and the elaborator

Everything today used only: inductive definitions, pattern matching, structural recursion, and induction. The simplest fragment of CIC already takes us far.

Next lecture: dependent types in practice.

Vectors, safe indexing with `Fin`, the gap between definitional and propositional equality in indexed types, and how to work with the `Prop` elimination restriction.